

The LLM Inference Platform Engineering Playbook — v5

A Principal-Level Production Systems Reference with Validatable Artifact Templates

Every snippet, manifest, and config in this document is intended as a validated shape, but must pass the profile-specific repo contract (§0.2) against a filled profile (§0.1) before use. The document defines the operating model and validation contract; the paired repository supplies the proof.

0. HOW TO READ THIS PLAYBOOK

0.1 Version lock — tested profiles (fill before adoption)

A bare version list warns about drift but gives no adoptable baseline. The unit of pinning is a **profile**: a named, dated, validated combination. Maintain at least one; every artifact in this playbook is re-validated against a profile, and the profile name appears in deployment annotations so incidents can be traced to a stack.

yaml

```
# versions.lock — one entry per validated profile. NO artifact ships against an unfilled
profiles:
- profile: h100-vllm-kserve          # canonical serving profile
  vllm: ""                          # flags, metric names, --speculative-config
  cuda: ""
  nvidia_driver: ""
  nccl: ""
  kubernetes: ""
  kserve: ""                        # LLMInferenceService CRD shape has churned
  gateway_api_inference_extension: ""
  dcgm_exporter: ""
  validated_on: ""                  # date the smoke + bench suite (§0.2) last passed
  validated_by: ""
- profile: sglang-structured         # add per-workload profiles as adopted
  sglang: ""
  validated_on: ""
```

Rule: no artifact enters a design doc or cluster without re-validation against its profile. Empty fields are a build-time failure, not a footnote.

0.2 Repo contract — what makes the templates executable

This document pairs with a repository; the document teaches, the repo proves:

```
text

/playbook
  /helm          # or /kustomize – the §5.2 manifest set, parameterized
  /benchmarks    # §2.5 protocol as scripts + trace-replay harness
  /dashboards    # Grafana JSON, PromQL panels from §2.4
  /runbooks      # §6.11 entries as paged-on documents
  /calculators   # §2.1 capacity script + §6.7 cost model as a notebook
  /ci            # eval-gate pipeline (§6.10), manifest lint, PromQL validation
versions.lock
Makefile
```

Minimum Makefile contract — these targets passing is the definition of "validated" in §0.1:

```
makefile

smoke-vllm:      # serve small model, run §1.2 smoke test, assert TTFT bound
deploy-kind:     # full §5.2 manifest set on a kind/minikube cluster (CPU mock ok)
bench-local:     # §2.5 warmup + steady-state + sweep, emit §2.5 report schema
validate-promql: # every §2.4 query parses and returns against a live scrape
lint-k8s:        # kubeconform/kube-linter over /helm output
test-gateway:    # alias translation, 429 paths, cancellation propagation
```

Failure-injection targets — untested degradation paths are assumptions, not designs. Each maps to a §6 mechanism and is exercised in game days:

```
makefile

test-brownout:   # drive KV pressure past each §6.5 rung; assert ladder actions
test-cancel-stream: # client disconnect mid-stream; assert engine abort + generated
test-tenant-quota: # over-budget tenant; assert 400/429 at admission, zero engine-
test-gpu-xid-sim: # inject simulated XID event; assert watcher cordons + drains w
test-kv-pressure: # heavy-context flood; assert heavy-pool routing + Critical res
test-rollback:   # canary with seeded eval regression; assert automated rollback
```

0.3 Claim-confidence legend

Every quantitative claim in this document carries one of four tags:

- **[Derived]** — follows from arithmetic stated inline; check the inputs.

- **[Cited]** — from a primary paper/spec; footnoted with source, date, config.
- **[Vendor]** — vendor-reported, vendor-optimal config; treat as ceiling, not expectation.
- **[Directional]** — workload-dependent; validate on your own traffic before relying on it.

Publication rule: [Vendor] and [Directional] claims are acceptable in an internal working document; before *external* publication every such claim is **resolved** (primary source + config located, tag upgraded) **or removed**. The claims register (Appendix B) is the worklist.

0.4 Scope and assumptions

- **Workloads covered:** chat, RAG, agentic/tool-calling, structured output, batch inference. Embeddings/rerankers and multimodal are noted where they change the design.
- **Traffic model:** you must know your prompt-length P50/P95/P99, output-length distribution, cache-hit potential (shared system prompts? multi-turn?), and request criticality mix. Every capacity and cost number downstream is a function of this distribution.
- **SLO model:** TTFT, TPOT/ITL, E2E latency, availability, and \$/1M tokens at SLO. Define cache-hit and cache-miss TTFT SLOs separately (§2.5) — prefix caching makes a single TTFT SLO unenforceable.
- **The MoE / extreme multi-node boundary, stated precisely.** This document covers *enough to decide*: whether you need EP at all (§5.1 tree), what the bottleneck will be (all-to-all bandwidth, §3.5), how to validate a fabric before trusting it (§5.4), and when disaggregation pays (§5.7). It defers *how to engineer the topology*: NVLink-domain planning, rail-optimized IB design, NCCL/RCCL tuning matrices, wide-EP routing, and expert-placement strategies — those are the committed companion document. If your decision requires the deferred material, treat this playbook as insufficient for that decision.

STAGE 1 — Mental Model, First Token, and the Gateway

1.1 The request-to-token lifecycle

```

HTTP ingress (gateway/LB) → authn/authz → ADMISSION CONTROL (token budget,
priority)
  → tokenization (text → token IDs)
  → scheduling/queueing (waiting queue, priority class)
  → PREFILL (all prompt tokens, one parallel pass, builds KV cache) ← compute-
bound
  → DECODE (1 token per forward pass, reads weights + full KV cache) ← memory-

```

bandwidth-bound

→ detokenization → streaming (SSE/WS) or JSON response

Note admission control appears *before* the engine. A platform that lets requests queue inside the engine until KV exhaustion triggers preemption storms has already failed; the gateway must reject (429 + `Retry-After`) what the fleet cannot serve within SLO. This is the single most common production incident pattern on shared LLM platforms (§6.1).

Metric	Definition	Driven by	Typical chat SLO
TTFT	arrival → first token	queue wait + prefill	P95 < 500 ms (set separately for cache hit/miss)
TPOT / ITL	time between output tokens	memory BW, decode batch	< 50 ms (≈20 tok/s perceived)
E2E	TTFT + (out_tokens-1)×TPOT	both	per use case
Throughput	tok/s, req/s	batch, saturation	—
Goodput	req/s meeting all SLOs	the production metric	capacity is sized from this

1.2 First token in <15 minutes — three paths

bash

PATH A – vLLM (production-aligned, OpenAI-compatible). Pin the version.
pip install "vllm==\${VLLM_VERSION}"
vllm serve microsoft/Phi-3-mini-4k-instruct \
 --dtype bfloat16 \
 --max-model-len 4096 \
 --gpu-memory-utilization 0.90 \
 --port 8000

curl http://localhost:8000/v1/chat/completions -H "Content-Type: application/json" -
 "model": "microsoft/Phi-3-mini-4k-instruct",
 "messages": [{"role":"user","content":"Explain KV cache in one sentence."}],
 "max_tokens": 64, "stream": true}'

bash

```
# PATH B — Ollama / llama.cpp (GGUF, CPU+GPU offload)
curl -fsSL https://ollama.com/install.sh | sh && ollama run tinyllama
# llama.cpp directly; -ngl = number of layers offloaded to GPU
./llama-server -m tinyllama-1.1b-chat.Q4_K_M.gguf -ngl 99 --port 8080
```

```
bash

# PATH C — SGLang (benchmark head-to-head later, §5)
pip install "sglang[all]==${SGLANG_VERSION}"
python -m sglang.launch_server --model-path microsoft/Phi-3-mini-4k-instruct --port
```

(HF TGI is in maintenance mode per Hugging Face's documentation [**Cited: HF docs, verify announcement date before quoting**] — use vLLM or SGLang for new builds.)

Smoke test (run after every deployment, gate readiness on it):

```
bash

# Expect: HTTP 200, non-empty choices[0], TTFT < 5s on a warm engine
time curl -sf http://localhost:8000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model": "microsoft/Phi-3-mini-4k-instruct", "messages": [{"role": "user", "content": "Hello"}]}' \
  | python3 -c "import sys,json; r=json.load(sys.stdin); assert r['choices'][0]['message']"
```

1.3 The inference gateway

⚠ **LEARNING ARTIFACT — DO NOT DEPLOY.** The Python gateway below exists to teach what a gateway *does*. It is single-process only (the in-memory round-robin state is per-worker under multi-worker ASGI, breaking load distribution), has no connection pooling limits, circuit breaking, retry budget, or memory bounds on streamed responses. For production, use **Envoy AI Gateway**, **kgateway** + **Gateway API Inference Extension** (§5.6), or **LiteLLM Proxy**. Hand-rolled async proxies under concurrent production load are a known failure class.

```
python
```

```

# gateway.py – LEARNING ARTIFACT. pip install fastapi uvicorn httpx
# Run single-worker only: uvicorn gateway:app --workers 1
import httpx, time
from fastapi import FastAPI, Request, HTTPException
from fastapi.responses import StreamingResponse

app = FastAPI()

# Endpoint management: client-facing alias → (upstream pool, real served model name)
# The alias MUST be translated before forwarding – vLLM rejects model names it wasn't
# started with (unless --served-model-name was set to the alias).
MODELS = {
    "phi3": {
        "upstreams": ["http://vllm-0:8000", "http://vllm-1:8000"],
        "served_name": "microsoft/Phi-3-mini-4k-instruct",
        "max_input_tokens": 4096, "max_output_tokens": 1024, # admission limits (§
    },
}
_rr = {k: 0 for k in MODELS} # single-process state; production uses EPP/external

def route(alias: str):
    m = MODELS.get(alias)
    if not m:
        raise HTTPException(404, f"unknown model alias: {alias}")
    _rr[alias] = (_rr[alias] + 1) % len(m["upstreams"])
    return m["upstreams"][_rr[alias]], m

@app.post("/v1/chat/completions")
async def chat(req: Request):
    body = await req.json()
    upstream, m = route(body.get("model", "phi3"))
    body["model"] = m["served_name"] # alias translation
    if body.get("max_tokens", 0) > m["max_output_tokens"]: # admission: output cap
        raise HTTPException(400, "max_tokens exceeds model policy")
    t0 = time.perf_counter()
    if body.get("stream"):
        async def relay():
            async with httpx.AsyncClient(timeout=None) as c:
                async with c.stream("POST", f"{upstream}/v1/chat/completions", json=
                    if r.status_code != 200: # propagate upstream fail
                        yield f'data: {{"error": "upstream {r.status_code}"}}\n\n'.e
                return
            first = True

```

```

        async for chunk in r.aiter_bytes():
            if first:
                print(f"TTFT={time.perf_counter()-t0:.3f}s upstream={ups
                first = False
            yield chunk
        return StreamingResponse(relay(), media_type="text/event-stream")
    async with httpx.AsyncClient(timeout=120) as c:
        r = await c.post(f"{upstream}/v1/chat/completions", json=body)
        if r.status_code != 200:
            raise HTTPException(r.status_code, r.text[:500])
        return r.json()

```

What a production gateway must own (the platform contract — each item maps to §6):
 authn/authz; model-alias translation; **admission control** (per-tenant token budgets, max
 input/output/context, priority classes); rate limiting with 429 + `Retry-After`; retries with
 budget + circuit breaking; streaming **cancellation propagation** (client disconnect must cancel
 the engine request, or you pay for abandoned generations); request size limits; usage metering
 per tenant; audit logging keyed by `X-Request-ID`; idempotency keys for retried non-streaming
 calls.

Production gateway	Use when
Envoy AI Gateway	Token-based rate limiting, multi-tenant quotas, upstream auth
kgateway / Gateway API Inference Extension	K8s-native, KV-cache- and LoRA-aware endpoint picking (§5.6)
LiteLLM Proxy	Multi-provider routing, budgets, virtual keys
NGINX / HAProxy	Plain L7 when nothing model-aware is needed

Streaming decision: SSE for chat (unidirectional, OpenAI standard, proxy-friendly);
 WebSockets only for duplex (voice, interruptible agents); non-streaming for batch/internal calls.
 Hardening: disable proxy buffering (`proxy_buffering off`), idle timeout > max generation
 time, propagate `X-Request-ID`, enforce response-size and stream-duration ceilings.

STAGE 2 — Measurement Before Tuning: Capacity Arithmetic, Profiling, Benchmarking

2.1 Capacity-planning arithmetic (heuristic with stated error bars — not "exact")

These formulas are planning estimates. Real memory depends on architecture (GQA/MQA/MLA, sliding-window layers), paged block slack, CUDA-graph capture, allocator fragmentation, activation peaks, LoRA adapters, draft models, and runtime implementation. **Apply a headroom factor and validate on your stack.** Use GiB consistently (1 GiB = 2^{30} bytes); marketing "GB" (10^9) inflates capacity ~7%.

```
WEIGHTS:      params × bytes/param
              70B × 2 (BF16) ≈ 140 GB ≈ 130 GiB | × 1 (FP8) ≈ 70 GB | × ~0.56
              (Q4_K_M ≈ 4.5 bpw) ≈ 39 GB
KV/token:     2 × n_layers × n_kv_heads × head_dim × bytes_per_elem
KV total:     Σ over running requests (seq_len_i × KV/token) ← not a clean
              batch×len product in practice
BUDGET:       weights + KV + activations + headroom
HEADROOM:     15-25% of total (allocator, CUDA graphs, sampler buffers,
              fragmentation, spikes)
```

Worked: Llama-3-70B (80 layers, 8 KV heads GQA, head_dim 128, BF16) **[Derived]**: KV/token = $2 \times 80 \times 8 \times 128 \times 2 = 327,680 \text{ B} \approx \mathbf{0.31 \text{ MiB/token}}$. 8 concurrent requests $\times 4,096 \text{ ctx} \approx \mathbf{10 \text{ GiB}}$. The same model with MHA (64 KV heads) would need ~80 GiB — the **8× GQA payoff** that makes dense-70B serving tractable. One 128K-context request alone $\approx 40 \text{ GiB KV}$.

The 70B fit question, corrected:

- BF16 weights $\approx 140 \text{ GB}$. An H200 (141 GB) technically *holds the weights* — and nothing else. No KV cache of useful size, no headroom, no CUDA graphs. **70B BF16 production serving requires $\geq 2 \times 80 \text{ GB GPUs}$, or quantization.** Single-H200 BF16 is a constrained-experiment configuration, not a production pattern.
- 70B FP8 ($\sim 70 \text{ GB}$) on one H100-80GB is *tight* ($\leq 10 \text{ GB}$ for KV + headroom $\rightarrow$ short contexts, low concurrency); one H200 leaves $\sim 60 \text{ GB}$ for KV and is a genuinely comfortable single-GPU production fit. **[Derived]**
- PagedAttention bounds *internal* fragmentation to roughly the last partially-filled block per sequence (low single-digit % **[Cited: vLLM/PagedAttention paper, SOSP'23]**), but block slack, padding, and dynamic batching still mean memory is never perfectly packed — hence the headroom factor.

Capacity planning helper — a *planning aid, not a source of truth*. Parameter count follows a source-of-truth ladder: (1) **safetensors index metadata** (actual checkpoint tensor sizes — correct for MoE, tied embeddings, gated MLPs, MLA, multimodal heads); (2) **hub model_info parameter count**; (3) architecture heuristic — *last resort only*, wrong for non-standard layouts. Production workflows inspect **local** artifacts (the checkpoint you'll actually serve, from your registry §6.9); the network fetch below is for exploration only — and fails with 401 on gated repos (most production models), which is itself the argument for the local path.

python

```

# capacity.py - PLANNING HELPER. python capacity.py meta-llama/Llama-3.1-70B-Instruct
# --dtype bf16 --kv-dtype fp8 --ctx 16384 --concurrency 32 --headroom 0.20
# Prefer --local-path to inspect the artifact you will actually serve.
import argparse, json, os, urllib.request

BYTES = {"fp32": 4, "float32": 4, "bf16": 2, "bfloat16": 2, "fp16": 2, "float16": 2,
        "fp8": 1, "int8": 1, "int4": 0.5}

def fetch_json(url):
    return json.load(urllib.request.urlopen(url))

def checkpoint_info(model, cfg, local_path=None):
    """Checkpoint BYTES are the serving-fit truth; parameter count is inferred
    only when the checkpoint dtype is unambiguous. For quantized checkpoints
    (FP8/INT4/AWQ/GPTQ/mixed), torch_dtype does NOT describe physical bytes/param -
    report size and skip param inference rather than mislead."""
    try:
        if local_path:
            idx = json.load(open(os.path.join(local_path, "model.safetensors.index.json")))
        else:
            idx = fetch_json(f"https://huggingface.co/{model}/raw/main/model.safetensors.index.json")
        total_bytes = idx["metadata"]["total_size"]
        quant = cfg.get("quantization_config")
        dtype = str(cfg.get("torch_dtype", "")).replace("torch.", "")
        if quant is None and dtype in BYTES:
            return total_bytes, total_bytes / BYTES[dtype], "safetensors-index"
        return total_bytes, None, "safetensors-index (quantized/mixed - params not inferred)"
    except Exception:
        pass

    # heuristic - WRONG for MoE/tied-embeddings/non-standard FFN; flagged in output
    h, L, V = cfg["hidden_size"], cfg["num_hidden_layers"], cfg["vocab_size"]
    params = 12 * L * h**2 + 2 * h * V
    return None, params, "HEURISTIC (verify!)"

if __name__ == "__main__":
    p = argparse.ArgumentParser()
    p.add_argument("model"); p.add_argument("--local-path", default=None)
    p.add_argument("--dtype", default="bf16"); p.add_argument("--kv-dtype", default="fp8")
    p.add_argument("--ctx", type=int, default=8192)
    p.add_argument("--concurrency", type=int, default=16)
    p.add_argument("--headroom", type=float, default=0.20)
    a = p.parse_args()
    cfg = (json.load(open(os.path.join(a.local_path, "config.json")))) if a.local_path else {}

```

```

        else fetch_json(f"https://huggingface.co/{a.model}/raw/main/config.json")
n_layers = cfg["num_hidden_layers"]; hidden = cfg["hidden_size"]
n_heads = cfg["num_attention_heads"]; n_kv = cfg.get("num_key_value_heads", n_h
head_dim = cfg.get("head_dim", hidden // n_heads)
params, source = None, ""
ckpt_bytes, params, source = checkpoint_info(a.model, cfg, a.local_path)
# Serving weight footprint: checkpoint bytes if serving the checkpoint as-is;
# params × serving-dtype bytes if re-quantizing/casting (requires known params)
if ckpt_bytes is not None:
    w_gib = ckpt_bytes / 2**30
    w_note = "checkpoint bytes (as-served)"
    if params is not None and a.dtype:
        w_gib = params * BYTES[a.dtype] / 2**30
        w_note = f"params x {a.dtype}"
else:
    w_gib = params * BYTES[a.dtype] / 2**30
    w_note = f"params x {a.dtype} (heuristic params)"
kv_tok = 2 * n_layers * n_kv * head_dim * BYTES[a.kv_dtype]
kv_gib = a.concurrency * a.ctx * kv_tok / 2**30
total = (w_gib + kv_gib) * (1 + a.headroom)
print(f"param source: {source}")
if ckpt_bytes is not None:
    print(f"checkpoint size: {ckpt_bytes/2**30:.1f} GiB")
print(f"weights≈{w_gib:.1f} GiB ({w_note}) kv/token={kv_tok/2**20:.2f} MiB "
      f"KV@{a.concurrency}x{a.ctx}≈{kv_gib:.1f} GiB TOTAL+headroom≈{total:.1f}")
print(f"fits: 1xH100-80? {total<80} 1xH200-141? {total<141} 2xH100? {total<160}")
print("PLANNING ESTIMATE – confirm against measured engine memory on your profil

```

2.2 Roofline and bottleneck classification

[Derived from public specs] H100 SXM5: ~989 TFLOPS dense FP16 / 3.35 TB/s HBM3 → ridge ≈ **295 FLOPs/byte FP16** (~591 FP8 at 1,979 TFLOPS). Prefill ≈ 200–400 FLOPs/byte → compute-bound, 90–95% util. Decode at low batch ≈ 1–2 FLOPs/byte — two orders of magnitude below the ridge → memory-bound, 20–40% util. **Decode throughput is set by HBM bandwidth, not TFLOPS.** Attention is memory-bound even in prefill — the motivation for FlashAttention (§3.4).

2.3 Profiling toolchain

```
bash
```

```
# Nsight Systems – system timeline (kernels, memcpy, NVTX, CPU)
nsys profile --trace=cuda,cudnn,cublas,osrt,nvtx \
  --pytorch=functions-trace --cudabacktrace=all --delay=60 -o llm_profile python inf
nsys stats llm_profile.nsys-rep
```

```
bash
```

```
# Nsight Compute – per-kernel roofline & bound classification
# GPU Speed-Of-Light: high SM%+low DRAM% = compute-bound; inverse = memory-bound
ncu --set roofline --kernel-name regex:attn --launch-count 10 -o kern python infer.p
```

```
python
```

```
# PyTorch Profiler
from torch.profiler import profile, ProfilerActivity
with profile(activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
             profile_memory=True, with_stack=True) as prof:
    model.generate(**inputs)
prof.export_chrome_trace("trace.json") # Perfetto / chrome://tracing
```

```
bash
```

```
# Live GPU telemetry
nvidia-smi dmon -s pucvmet -d 2
# DCGM profiling fields (verify IDs against your DCGM release):
# 1001 GR_ENGINE_ACTIVE | 1002 SM_ACTIVE | 1004 TENSOR_ACTIVE | 1005 DRAM_ACTIVE (me
dcgmi dmon -e 1001,1002,1004,1005 -d 1000
```

Bottleneck classifier: DRAM_ACTIVE high + TENSOR low → memory-bound (decode) → §4 memory recipes. TENSOR/SM high → compute-bound (prefill) → compute recipes. Both low + queue growing → host/scheduler/network bound — fix CPU-side code before touching the GPU.

2.4 Observability stack

```
yaml
```

```
# prometheus.yml
scrape_configs:
  - job_name: vllm
    scrape_interval: 5s
    static_configs: [{ targets: ["vllm-0:8000", "vllm-1:8000"] }]
  - job_name: dcgm
    static_configs: [{ targets: ["dcgm-exporter:9400"] }]
```

```
bash
```

```
# DCGM exporter (verify chart URL against current NVIDIA docs)
helm repo add gpu-helm-charts https://nvidia.github.io/dcgm-exporter/helm-charts
helm install dcgm gpu-helm-charts/dcgm-exporter -n monitoring
```

PromQL panels (names are profile-pinned; examples reflect current vLLM V1 docs — V1 renamed the per-token histogram to `inter_token_latency`):

```
promql
```

```
histogram_quantile(0.95, rate(vllm:time_to_first_token_seconds_bucket[5m]))
histogram_quantile(0.95, rate(vllm:inter_token_latency_seconds_bucket[5m]))
vllm:num_requests_running      vllm:num_requests_waiting      vllm:kv_cache_usage_perc
rate(vllm:prefix_cache_hits[5m]) / rate(vllm:prefix_cache_queries[5m])
DCGM_FI_PROF_DRAM_ACTIVE
```

Alert starter set: TTFT P95 > SLO 5m; `waiting > 2×running` 5m (queue-bound);
`kv_cache_usage_perc > 0.92` (preemption imminent — triggers brownout ladder §6.5);
`DCGM_FI_DEV_XID_ERRORS > 0` (GPU fault — drain node, §5.2 probes). Add OpenTelemetry tracing gateway→engine to attribute latency between queue wait and prefill.

2.5 Benchmarking methodology (protocol, not vibes)

Protocol — fixed acceptance criteria:

1. **Capture your distribution:** prompt-length P50/P95/P99, output-length distribution, cache-hit potential, structured-output fraction, from production traces where they exist. Synthetic-vs-production mismatch is the #1 source of wrong capacity numbers.
2. **Warmup 10 min** (CUDA graph capture, torch.compile, prefix-cache population) — never measure cold.
3. **Steady state ≥30 min** per concurrency level.
4. **Sweep request rate** until SLO goodput peaks then declines — that knee is the per-replica capacity number feeding autoscaling (§5.3) and cost (§6.7).

5. **Report:** TTFT P50/P95/P99 (split cache-hit vs cache-miss), TPOT P50/P95/P99, E2E, queue wait, preemption count, KV usage, DRAM/TENSOR active, \$/1M tokens at SLO.
6. **Re-run on every engine upgrade and quantization change;** gate promotion on no-regression (§6.10).

bash

```
# vLLM built-in (verify subcommand exists in your pinned release)
vllm bench serve --backend openai-chat --base-url http://localhost:8000 \
  --model microsoft/Phi-3-mini-4k-instruct \
  --dataset-name sharegpt --dataset-path ShareGPT_V3_unfiltered.json \
  --num-prompts 1000 --request-rate 8 --max-concurrency 64

# genai-perf / AIPerf – goodput with SLO constraints
genai-perf profile -m phi3 --service-kind openai --endpoint-type chat \
  --synthetic-input-tokens-mean 512 --output-tokens-mean 256 \
  --concurrency 32 --goodput time_to_first_token:500 inter_token_latency:50
```

Prefix caching ⇒ **split SLOs**. A cache hit skips prefill (TTFT $\approx$ queue + first decode step); a miss pays full prefill. One blended TTFT SLO is unenforceable — define **TTFT-hit** and **TTFT-miss** SLOs and monitor the hit rate as a first-class signal (it is also a cost driver, §6.7).

Canonical benchmark report schema — every `bench-local` run emits this; the filled report is the evidence artifact that turns sizing math into adopted numbers:

yaml

```

benchmark_report:
  profile: h100-vllm-kserve          # §0.1 profile this ran against
  model: ""                          # exact revision/quantization
  hardware: ""                       # GPU SKU × count, interconnect class
  traffic:
    source: ""                       # production trace | synthetic (state which)
    prompt_tokens: { p50: 0, p95: 0, p99: 0 }
    output_tokens: { p50: 0, p95: 0, p99: 0 }
    cache_hit_rate: 0.0
    structured_output_fraction: 0.0
  results:
    ttft_ms: { p50: 0, p95: 0, p99: 0, hit_p95: 0, miss_p95: 0 }
    tpot_ms: { p50: 0, p95: 0, p99: 0 }
    e2e_s:    { p50: 0, p95: 0, p99: 0 }
    goodput_knee: { concurrency: 0, output_tok_s: 0, input_tok_s: 0 }
    queue_wait_ms_p95: 0
    preemptions: 0
    kv_usage_peak: 0.0
    dram_active_mean: 0.0
  decisions:                          # the numbers this report justifies
    per_replica_concurrency_cap: 0
    replica_count: 0                  # via §5.3 Little's Law derivation
    keda_queue_threshold: 0
    cost_per_1M: { input: 0.0, output: 0.0, total: 0.0 } # §6.7
  run: { warmup_min: 10, steady_min: 30, date: "", operator: "" }

```

STAGE 3 — Engine Internals (and Building One)

3.1 Minimal engine from scratch

```

python

# The decode loop at the heart of every engine
out = model(input_ids, use_cache=True)          # PREFILL: one parallel pass
past = out.past_key_values
tok_id = out.logits[:, -1, :].argmax(-1, keepdim=True)
for _ in range(max_new_tokens):                 # DECODE: one token per pass
    out = model(tok_id, past_key_values=past, use_cache=True)
    past = out.past_key_values                   # KV cache grows by 1 token
    tok_id = sample(out.logits[:, -1, :])        # greedy / top-p / temperature
    if tok_id.item() == eos: break

```

Two leaps to a real engine: **continuous batching** (re-evaluate the batch every iteration — admit/retire at token granularity) and **paged KV** (no contiguous max-length pre-allocation).

```
python
```

```
# Iteration-level scheduler skeleton (the Orca/vLLM idea)
while True:
    finished = [s for s in running if s.done]; running -= set(finished)
    free_blocks += release(finished)
    # Admission inside the engine is the LAST line of defense; the gateway (§6.1)
    # should have already enforced budgets. Engine-side: admit only if KV fits.
    while waiting and fits(waiting[0], free_blocks):
        running.add(waiting.pop(0))
    step_batch = build_batch(running)          # mixed prefill+decode (chunked prefill)
    logits = model.forward(step_batch)        # ONE fused forward for everyone
    append_tokens(running, sample(logits))
    # Under KV pressure: preempt lowest-priority sequences (swap to CPU or recompute)
    # preemption is the engine's overload behavior; visible as vllm:num_preemptions.
```

Build this against a 1B model and you understand 80% of vLLM's scheduler — including why preemption policy and admission order are *platform* decisions, not engine trivia.

3.2 vLLM internals

- **PagedAttention** [Cited: Kwon et al., SOSP '23, arXiv:2309.06180]: KV in fixed blocks (16 tokens default); per-sequence block tables map logical→physical (OS-paging analogy); copy-on-write prefix sharing; internal fragmentation bounded to the last partial block. Paper reports 2-4× throughput vs FasterTransformer/Orca at equal latency (paper config).
- **Continuous batching**: waiting/running/swapped queues; token-granular admission; preemption by recompute or swap when KV exhausts.
- **V1 engine** (default in 2025-era releases — verify your pin): zero-copy DMA, pinned host memory, restructured scheduler, independent prefill/decode handling.
- **Chunked prefill** (Sarathi lineage): long prefills split into `--max-num-batched-tokens` chunks, piggybacked with decodes → protects ITL from long-prompt interference.
- Prefix caching, piecewise CUDA graphs, torch.compile, FlashAttention/FlashInfer kernel stack.

3.3 SGLang internals

- **RadixAttention** [Cited: SGLang paper/LMSYS report, Jan 2024]: KV cache in a radix tree keyed by token sequence; matched prefixes cost zero prefill; **LRU eviction** over tree nodes under memory pressure — note that under severe pressure, eviction of hot shared prefixes

converts cache-hit traffic to cache-miss TTFT, which is why hit-rate monitoring matters (§2.5). Up to $6.4\times$ on prefix-heavy workloads vs then-current vLLM (paper config; vLLM has since added prefix caching — re-benchmark, [**Directional**]).

- **Jump-forward decoding** + xGrammar: constrained decoding via compressed FSM emits multi-token deterministic spans ($\sim 2\times$ latency win on structured output, [**Vendor: SGLang materials — resolve per Appendix B**]). Note grammar compilation and FSM evaluation add **CPU-side cost** — structured output shifts bottlenecks host-ward at high QPS (§6.4).
- **Overlap scheduler**: CPU scheduling overlapped with GPU compute (vendor-reported utilization gains, [**Vendor**]).
- **sgl-router**: routes on radix-tree cache state fleet-wide.

3.4 FlashAttention — what it actually does

Never materializes the $N\times N$ attention matrix in HBM; tiles Q/K/V into SRAM, computes **online softmax** incrementally. FA2 = better warp-level parallelism. **FA3** (Hopper): producer/consumer **warp specialization** with TMA + FP8 — paper reports $\sim 75\%$ H100 FP16 utilization (~ 740 TFLOPS) and ~ 1.2 PFLOPS FP8, vs $\sim 35\%$ for FA2 [**Cited: Shah et al., arXiv:2407.08608 — benchmark-specific**]. **Kernel-path selection is version-sensitive**, the same way CRDs and flags are: which attention backend an engine picks depends on engine release, FlashAttention/FlashInfer build, GPU arch, head dim, dtype, and feature flags (e.g., FP8 KV or sliding window can route to a different kernel). Confirm the active backend per profile (vLLM logs the selected attention backend at startup) rather than assuming FA3.

3.5 Parallelism primitives — the communication table (corrected)

Megatron-style TP on a standard transformer block requires communication after **both** the attention output projection **and** the MLP down projection:

Primitive	What is sharded	Communication per layer (forward)	Link sensitivity
TP (tensor)	QKV/MLP weight matrices (column→row pattern)	$2 \times$ all-reduce (after attn out-proj, after MLP down-proj) [Cited: Megatron-LM, Shoeybi et al.]	Extreme — every layer, every step. NVLink-class only
TP + SP (sequence parallel)	+ activations along sequence in norm/dropout regions	all-reduce decomposed into reduce-scatter + all-gather (same volume, less activation memory)	Same as TP
PP (pipeline)	Layer ranges across stages	point-to-point activation transfer per microbatch; bubble $\approx (PP-1)/(microbatches+PP-1)$ [Derived]	Tolerant — works across nodes/Ethernet
DP (replicas)	Nothing (full copies)	none at inference	None — the default throughput scaler
EP (expert, MoE)	Experts across GPUs	all-to-all token dispatch + combine per MoE layer — the real MoE bottleneck	High — wants NVLink/IB; bandwidth $\times$ token imbalance

Interconnect reality [Derived from specs]: H100 NVLink4 $\approx$ 900 GB/s/GPU bidirectional; PCIe Gen5 x16 $\approx$ 128 GB/s — a $7\times$ gap. TP across PCIe-only GPUs usually loses to PP or a bigger single GPU. RCCL/xGMI on AMD has different topology constraints — validate per §5.4.

3.6 Engine selection matrix

Engine	Core tech	Reach for it when
vLLM	PagedAttention, V1 scheduler	Default: broad models, OpenAI API, LoRA, quant formats
SGLang	RadixAttention, jump-forward, overlap sched	Agents, multi-turn, few-shot, structured output
TensorRT-LLM	AOT compiled engines, in-flight batching, FP8/FP4	Peak NVIDIA throughput; you accept compile + lock-in
llama.cpp / Ollama	GGUF, CPU/GPU layer offload	Local, edge, air-gapped, consumer GPUs

Engine	Core tech	Reach for it when
NVIDIA Dynamo	Disaggregated P/D, KV-aware routing, multi-node	Cluster-scale over vLLM/SGLang/TRT-LLM backends
LMDeploy / DeepSpeed-MII	TurboMind; Mooncake PD integration	Alternative runtimes, specific fits

STAGE 4 — Optimization (Decision Contracts, Not Tips)

Every optimization below carries a **decision contract**: precondition (measured), expected metric movement, cost/failure mode, rollback. Apply nothing without its precondition.

4.0 The decision framework

Measured bottleneck	Symptoms	Apply (in order)
Memory-bound (decode)	DRAM_ACTIVE high, TENSOR low	FP8 KV cache → weight quant (FP8 → AWQ/GPTQ INT4) → larger batch → prefix caching → KV offload
Compute-bound (prefill)	TENSOR/SM high	Chunked-prefill tuning → FP8 compute → CUDA graphs/torch.compile → disaggregation
Latency-bound (low batch)	SLO miss at low QPS	Speculative decoding (verify α) → higher-bandwidth GPU → smaller/quantized model
Queue-bound	waiting » running, TPOT stable	Admission control first (§6.1), then replicas + autoscaling (§5.3)

4.1 Production vLLM config template

```
bash
```

```
# Comments above flags – backslash must be the last character on a continued line.
# TP across an NVLink pair:
#   --tensor-parallel-size 2
# FP8 KV cache: ~2x KV capacity → larger feasible batch (precondition: Hopper/Ada+,
#   --kv-cache-dtype fp8
# Cap context to the product's real need – the single biggest KV lever:
#   --max-model-len 16384
# Chunked-prefill token budget: raise = throughput, lower = better ITL:
#   --max-num-batched-tokens 8192
vllm serve meta-llama/Llama-3.1-70B-Instruct \
  --tensor-parallel-size 2 \
  --dtype bfloat16 \
  --quantization fp8 \
  --kv-cache-dtype fp8 \
  --max-model-len 16384 \
  --max-num-seqs 256 \
  --max-num-batched-tokens 8192 \
  --enable-prefix-caching \
  --enable-chunked-prefill \
  --gpu-memory-utilization 0.90 \
  --port 8000
```

4.2 Quantization — kernel coverage, pipeline, and eval gates

FP8 capability is not one thing [corrected]:

Arch	SM	FP8 status
Ada (L40S, RTX 4090)	8.9	FP8 instructions present; kernel coverage in vLLM/TRT-LLM is narrower than Hopper — validate your exact model/kernel path
Hopper (H100/H200)	9.0	Full FP8 inference path; the recommended starting point
Blackwell (B200/GB200)	10.0	FP8 + FP4 (NVFP4); mixed precision (FP4 weights, FP8/BF16 attention) is the production pattern [Vendor]

Checkpoint format is a second compatibility axis (compressed-tensors vs fbgemm-fp8 vs TRT-LLM engines) — match format to engine.

```
python
```

```
# FP8 W8A8 with llm-compressor (vLLM-native)
from llmcompressor.transformers import oneshot
from llmcompressor.modifiers.quantization import QuantizationModifier
oneshot(model="meta-llama/Llama-3.1-70B-Instruct",
        recipe=QuantizationModifier(targets="Linear", scheme="FP8_DYNAMIC",
                                     ignore=["lm_head"]),
        output_dir="Llama-3.1-70B-FP8")
# then: vllm serve ./Llama-3.1-70B-FP8
```

Method	Scheme	Use when	Watch for
FP8 dynamic	W8A8 float	Hopper/Blackwell default; usually safe	format/engine match
SmoothQuant	W8A8 INT	Broad HW throughput	activation outliers
AWQ / GPTQ	W4A16	VRAM-constrained decode	needs domain eval — benchmark-invisible regressions are common
NVFP4	4-bit float	Blackwell	mixed precision pattern; [Vendor] accuracy claims
GGUF Q4_K_M	~4.5 bpw	llama.cpp local	quality cliff is Q3→Q4

Eval gate with teeth (a benchmark-clean quant can still crater tool-calling):

```
bash

# Necessary baseline:
lm_eval --model vllm --model_args pretrained=./Llama-3.1-70B-FP8 \
  --tasks gsm8k,mmlu,ifeval --batch_size auto
# Sufficient gate adds: long-context (RULER), tool/function-calling (BFCL),
# your domain corpus, and PRODUCTION TRAFFIC REPLAY (sampled, anonymized).
# Promotion rule: block if any tracked metric regresses beyond the error budget (§6.
```

4.3 Speculative decoding — decision contract

Precondition: latency-bound at low/medium batch; measured acceptance rate $\alpha \geq \sim 0.6$ on *your* traffic. **Failure mode:** at $\alpha < 0.5$ or at high batch, verification overhead reduces throughput — and production batch sizes fluctuate, so a config that wins at 3 a.m. can lose at peak. **Mitigation:**

isolate speculative decoding to dedicated low-latency pools, or toggle on load signals, rather than enabling fleet-wide. **Rollback:** remove the flag; no state.

```
bash

# EAGLE-3 in vLLM (schema is version-sensitive – verify against your pin)
vllm serve meta-llama/Llama-3.1-70B-Instruct --tensor-parallel-size 2 \
  --speculative-config '{"method":"eagle3","model":"yuhuili/EAGLE3-LLaMA3.1-Instruct

# n-gram / prompt-lookup: zero extra model – strong for RAG/summarization where output
vllm serve MODEL \
  --speculative-config '{"method":"ngram","num_speculative_tokens":4,"prompt_lookup_
```

EAGLE-3 paper reports ~3–6.5× over autoregressive at low batch [Cited: Li et al., EAGLE-3 — paper config; production gains are typically lower and workload-dependent [Directional]].

4.4 KV-cache management ladder

1. `--max-model-len` capped to real product need (cheapest win)
2. `--kv-cache-dtype fp8` (eval-gated)
3. Prefix caching (vLLM) / RadixAttention (SGLang default) — then **split TTFT SLOs** (§2.5) and decide the multi-tenant cache policy (§6.2: shared prefixes are a chargeback and isolation question, not just a perf feature)
4. CPU/NVMe offload + cross-instance reuse: **LMCache** (connector names/roles are version-sensitive)
5. Cluster KV pooling: **Mooncake** / **NIXL** (§5.7)

4.5 Hardware decision table (inputs first, labels removed)

Choosing hardware requires: model + dtype, context P95/P99, output length, TTFT/TPOT SLO, QPS, batchability, cache-reuse rate, interconnect available, region, and price. The table gives physical envelopes — [Derived from public specs] — not verdicts:

GPU	Mem / BW	Physical envelope
RTX 4090 / 5090	24-32 GB / ~1 TB/s	dev, ≤13B, local INT4; FP8 on Ada = limited kernels
L40S	48 GB / 0.86 TB/s	≤34B INT4/FP8, cost-efficient inference
A100 80GB	80 GB / 2 TB/s	mature fleets; 70B BF16 = 2×
H100 80GB	80 GB / 3.35 TB/s	70B FP8 single-GPU is <i>tight</i> (short ctx, low concurrency); 70B BF16 = 2×
H200	141 GB / 4.8 TB/s	70B FP8 single-GPU comfortable; 70B BF16 single-GPU = weights-only, not production; highest Hopper decode bandwidth
B200	192 GB / ~8 TB/s	Blackwell FP4/FP8; verify per-SKU capacity
GB200 NVL72	rack-scale, NVLink5	frontier MoE; aggregate figures are [Vendor]
MI300X	192 GB / 5.3 TB/s	largest single-GPU HBM class; software maturity gap is real but moving fast — re-benchmark on current ROCm/vLLM before deciding [Directional]
MI325X	256 GB / 6.0 TB/s	same caveat; split from MI300X deliberately

Buying rule: memory-bound serving buys **bandwidth** + **capacity**; prefill-heavy buys FP8/FP4 compute. Fit check via the §2.1 calculator with headroom. TP only within an NVLink/xGMI domain.

STAGE 5 — Scale: Kubernetes, Sizing Math, Fabric, Disaggregation

5.1 Parallelism decision tree

```

Does the model (quantized) fit on ONE GPU with KV + headroom (§2.1 calculator)?
├─ YES → DP replicas. Scale out. Stop adding parallelism.
└─ NO → Does it fit in ONE NODE (NVLink/xGMI domain)?
    ├─ YES → TP = smallest power of 2 that fits. + DP for throughput.
    └─ NO → TP within node × PP across nodes.

```

MoE? → TP/DP for dense layers + EP for experts;
your bottleneck is now all-to-all bandwidth, not FLOPS.
Disaggregate prefill/decode only on measured interference or SKU economics (§5.7).

Workload × interconnect → topology matrix (compact form of the tree, indexed by the two inputs that actually decide it):

Prompt-length profile	NVSwitch/NVLink node	PCIe-only node	Multi-node IB/RoCE	Multi-node Ethernet
Short prompts, chatty (P95 < 2K)	DP replicas; TP only if model doesn't fit	DP replicas of quantized single-GPU fits — avoid TP	DP across nodes	DP across nodes
Mixed (P95 2K-16K)	TP-to-fit + DP; chunked prefill on	smallest quantization that fits one GPU, else PP	TP in-node × DP across	same; never TP across the wire
Long-context heavy (P95 > 32K)	TP + heavy-pool isolation (§6.2); disaggregation candidate	wrong hardware class — re-plan	TP in-node × PP across; disaggregated P/D pools	PP only; expect TTFT pain, set SLOs accordingly
MoE	TP dense + EP experts in NVLink domain	not viable at scale	EP across nodes — all-to-all is the budget	not viable at scale

5.2 Production vLLM on Kubernetes (completed manifest set)

yaml


```

# --- deployment.yaml ---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vllm-llama70b
  labels: { app: vllm-llama70b }
  annotations: { platform/profile: h100-vllm-kserve } # ties pods to a versions.l
spec:
  replicas: 2
  selector: { matchLabels: { app: vllm-llama70b } }
  template:
    metadata:
      labels: { app: vllm-llama70b }
      annotations: { prometheus.io/scrape: "true", prometheus.io/port: "8000" }
    spec:
      # Non-root requires an explicit UID/GID AND a writable, non-root cache path -
      # runAsNonRoot with the default /root/.cache/huggingface mount will fail.
      securityContext:
        runAsNonRoot: true
        runAsUser: 1000
        runAsGroup: 1000
        fsGroup: 1000
        seccompProfile: { type: RuntimeDefault }
      containers:
        - name: vllm
          # PINNED image - never :latest in production
          image: vllm/vllm-openai:vX.Y.Z
          args: ["--model", "meta-llama/Llama-3.1-70B-Instruct",
            "--tensor-parallel-size", "2", "--quantization", "fp8",
            "--kv-cache-dtype", "fp8", "--max-model-len", "16384",
            "--enable-prefix-caching", "--enable-chunked-prefill"]
          env:
            - name: HF_HOME # cache path matching the non-root user
              value: /models/hf-cache
            - name: HF_TOKEN
              valueFrom: { secretKeyRef: { name: hf-token, key: token } }
          resources: { limits: { nvidia.com/gpu: 2 } }
          ports: [{ containerPort: 8000 }]
          # PROBE DESIGN (division of labor - each probe does one job):
          # startupProbe: absorbs multi-minute weight load AND proves the CUDA context
          # works ONCE via a real micro-generation. Uses /v1/chat/completions (the
          # endpoint instruct models actually serve) and python for JSON validation
          # no jq-in-image assumption. Expensive checks belong here.

```

```

startupProbe:
  exec:
    command: ["sh", "-c",
      "curl -sf -m 30 http://localhost:8000/v1/chat/completions -H 'Content-
periodSeconds: 15
failureThreshold: 60          # up to 15 min for pull + load + first gen
readinessProbe:
  httpGet: { path: /health, port: 8000 }
  periodSeconds: 10
# livenessProbe: CONSERVATIVE – /health only. A per-minute real generation
# probe (a) adds load, (b) queues behind tenant traffic under saturation and
# times out, killing a healthy-but-busy pod (priority starvation). If you do
# run a generation liveness probe, the engine must serve it at highest
# internal priority – most engines cannot guarantee that today.
livenessProbe:
  httpGet: { path: /health, port: 8000 }
  periodSeconds: 30
  failureThreshold: 5
volumeMounts: [{ name: model-cache, mountPath: /models/hf-cache }]
lifecycle:
  # DRAIN CONTRACT – sleep is a placeholder for the real sequence, which is:
  #   preStop fires → pod marks itself draining (fail readiness or call /dra
  #   if the engine exposes one) → EndpointSlice goes NotReady → gateway sto
  #   routing NEW requests → in-flight streams complete within the grace bud
  #   → process exits. A bare sleep only buys time; it does not PROVE the
  #   gateway stopped routing – verify the sequence in test-cancel-stream (§
  preStop: { exec: { command: ["sh", "-c", "sleep 30"] } }
# On-demand: 180s drain budget. SPOT NODES: the cloud interruption notice caps
# your real budget (AWS gives 120s) – preStop + drain MUST complete inside the
# notice window, so spot pools need terminationGracePeriodSeconds <= notice
# and shorter max-stream durations. A grace period the cloud won't honor is fi
terminationGracePeriodSeconds: 180
volumes:
- name: model-cache
  persistentVolumeClaim: { claimName: hf-cache }
nodeSelector: { nvidia.com/gpu.product: NVIDIA-H100-80GB-HBM3 }
tolerations:
- { key: nvidia.com/gpu, operator: Exists, effect: NoSchedule }
topologySpreadConstraints:          # replicas across failure domains
- maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
whenUnsatisfiable: ScheduleAnyway
labelSelector: { matchLabels: { app: vllm-llama70b } }

```

```

# --- service.yaml ---
apiVersion: v1
kind: Service
metadata: { name: vllm-llama70b }
spec:
  selector: { app: vllm-llama70b }
  ports: [{ port: 8000, targetPort: 8000 }]
---
# --- pvc.yaml --- (see cache-strategy note below before assuming RWX)
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: hf-cache }
spec:
  accessModes: ["ReadWriteMany"]
  resources: { requests: { storage: 500Gi } }
---
# --- pdb.yaml ---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: { name: vllm-llama70b }
spec:
  minAvailable: 1
  selector: { matchLabels: { app: vllm-llama70b } }
---
# --- networkpolicy.yaml --- engine pods accept traffic ONLY from the gateway
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: vllm-llama70b-ingress }
spec:
  podSelector: { matchLabels: { app: vllm-llama70b } }
  policyTypes: [Ingress]
  ingress:
    - from:
      - podSelector: { matchLabels: { role: inference-gateway } }
      # kubernetes.io/metadata.name is auto-applied by K8s 1.21+; custom labels require
      # explicitly labeling the namespace
      - namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: monitoring } }
    ports: [{ port: 8000 }]

```

GPU-fault detection is a platform component, not a probe. The wedged-CUDA-context problem (HTTP 200 while the GPU is dead) is solved by a **DCGM/XID watcher**: alert + automated cordon/drain on `(DCGM_FI_DEV_XID_ERRORS > 0)` (NVIDIA's GPU operator health

checks, or a DaemonSet watching XID events). The startup probe proves the context once; the watcher catches it dying later — without generating synthetic load every minute on every pod.

Model-cache strategy — RWX is one option, not the default. RWX PVCs require a compatible storage class (NFS/CephFS/Filestore) many GPU clusters lack. Alternatives, in rough order of preference: **(a)** bake/pull model as an **OCI artifact** (KServe model-cache, ORAS) — immutable, signed (§6.9), CDN-able; **(b)** per-node **hostPath cache pre-warmed by a DaemonSet** — fastest cold start, node-local; **(c)** RWO PVC per replica — simple, duplicates storage; **(d)** RWX shared cache — convenient where the storage class exists. Choose per cluster; the manifest's PVC is a placeholder for that decision.

Zero-trust note: the manifest serves bare HTTP on 8000. Regulated environments require **mTLS between gateway and engine** — typically via a service mesh (Istio/Linkerd sidecars or ambient) rather than engine-level TLS. Treat the NetworkPolicy above as the floor and mesh mTLS as the regulated-environment standard (§6.8).

5.3 Autoscaling — sizing math first, KEDA second

Derive the threshold; don't guess it. Little's Law: $L = \lambda \times W$ (in-system requests = arrival rate $\times$ time-in-system).

From the §2.5 benchmark at the goodput knee:

W = $E[TTFT] + E[\text{output_tokens}] \times E[TPOT]$ (mean service time at knee)

C_{replica} = concurrency at knee (bounded by KV budget: $KV_{\text{total}} / E[\text{KV per request}]$)

$\text{replicas} = \text{ceil}(\lambda_{P99} \times W / (C_{\text{replica}} \times \text{safety}))$, $\text{safety} \approx 0.7-0.8$

KEDA queue threshold $\approx C_{\text{replica}} \times (1 - \text{safety})$ (queue forming past headroom)

Worked [Derived]: knee at $C_{\text{replica}} = 24$, $W = 12$ s, expected peak $\lambda = 6$ req/s $\rightarrow$ in-flight = 72 $\rightarrow$ replicas = $\text{ceil}(72 / (24 \times 0.75)) = 4$, queue threshold $\approx 24 \times 0.25 = 6$ per replica.

yaml

```

apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata: { name: vllm-scaler }
spec:
  scaleTargetRef: { name: vllm-llama70b }
  minReplicaCount: 2
  maxReplicaCount: 12           # from sizing math + budget, not vibes
  cooldownPeriod: 600           # GPU pods churn slowly; flapping is expensive
  triggers:
  - type: prometheus
    metadata:
      serverAddress: http://prometheus:9090
      query: sum(vllm:num_requests_waiting{app="vllm-llama70b"}) / count(vllm:num_re
      threshold: "6"            # derived above – per-replica queue headroom

```

Reactive autoscaling cannot save TTFT during a spike. Scrape interval + KEDA loop + pod schedule + multi-minute weight load = the SLO is blown long before the new replica serves. Production pattern:

1. **Admission control absorbs the spike** (§6.1): shed Sheddable-class traffic with 429 + `Retry-After` the moment the fleet passes its goodput knee — protect Critical-class SLOs.
2. **Warm headroom:** run N+1 with pre-pulled weights (PVC/OCI cache makes "warm" minutes→seconds); for bursty products keep a scaled-to-zero-but-image-warm pool.
3. **Predictive scaling** where traffic has diurnal shape (scale on schedule or token-velocity trend, not just instantaneous queue). Autoscaling reclaims cost at the timescale of tens of minutes; it is not a latency-protection mechanism.

5.4 Fabric & topology validation (gate, don't hope)

TP=8 over a non-NVSwitch topology silently halves throughput; a mis-mapped IB rail does the same across nodes. **Validate before first serve, and after any driver/firmware change:**

```
bash
```

```
# 1. Topology audit – expect NV# (NVLink) between TP peers, not PHB/PIX (PCIe)
nvidia-smi topo -m

# 2. NCCL all-reduce busbw across the intended TP/PP group
git clone https://github.com/NVIDIA/nccl-tests && cd nccl-tests && make
./build/all_reduce_perf -b 1G -e 8G -f 2 -g 8
# GATE: busbw ≥ ~80% of the topology's theoretical (e.g., NVSwitch HGX H100 ≈ 450+ G
# busbw expected). If you're at 50%, STOP and debug topology before benchmarking the

# 3. Multi-node: verify rail alignment and RDMA
# NCCL_IB_HCA pinned to the correct HCAs; GPUDirect RDMA active (no host bounce)
NCCL_DEBUG=INFO NCCL_IB_HCA=mlx5_0,mlx5_1 ./build/all_reduce_perf -b 1G -e 8G -g 16
# Inspect NCCL_DEBUG output: "via NET/IB/GDRDMA" good; "via NET/Socket" = silent dead

# 4. AMD: rccl-tests with xGMI topology; different constraints – validate separately
```

Record passing busbw numbers per topology class in the runbook; future incident triage compares against them.

5.5 Multi-node 405B-class serving

```
bash

# vLLM multi-node: TP within node (NVLink), PP across nodes (verify script path in y
# Node 0:
bash examples/online_serving/run_cluster.sh vllm/vllm-openai HEAD_IP --head
# Node 1..N: same with --worker; then on head:
vllm serve meta-llama/Llama-3.1-405B-Instruct \
  --tensor-parallel-size 8 \
  --pipeline-parallel-size 2 \
  --quantization fp8 \
  --kv-cache-dtype fp8 \
  --max-model-len 8192
```

Pipeline bubble $\approx (PP-1)/(microbatches+PP-1)$ **[Derived]** — keep PP small and microbatches high. MoE (DeepSeek-class): EP for experts; the all-to-all dispatch is the bottleneck — size IB/NVLink for it and monitor token imbalance across experts.

5.6 K8s control plane: KServe, Inference Extension, Ray Serve

```
yaml
```

```
# KServe LLMInferenceService — CRD shape is version-sensitive; validate against your
apiVersion: serving.kserve.io/v1alpha1
kind: LLMInferenceService
metadata: { name: llama70b }
spec:
  model: { uri: "oci://registry.internal/models/llama-3.1-70b-fp8:SIGNED_TAG", name:
  replicas: 2
  router: { route: {}, gateway: {}, scheduler: {} }
```

Note the model URI: **private OCI registry with a signed tag** (§6.9) — not a runtime pull from the public internet.

Gateway API Inference Extension / Endpoint Picker (EPP): routes on a scoring function over live signals — KV-cache utilization, queue depth, LoRA-adaptor locality, prefix-cache affinity — plus request **criticality** (Critical vs Sheddable; sheddable dropped under saturation). This is the production replacement for the round-robin logic in §1.3, and the enforcement point for §6.1 priorities. Behavior depends on the gateway/EPP implementation — verify against your pinned release.

Ray Serve (API evolves — verify): choose it for many-model fleets, fractional/heterogeneous GPU packing, and Python-level composition; choose KServe/llm-d for K8s-native CRDs + gateway integration; KubeRay gives both.

```
python

from ray import serve
from ray.serve.llm import LLMConfig, build_openai_app
config = LLMConfig(
    model_loading_config={"model_id": "llama-70b",
                          "model_source": "meta-llama/Llama-3.1-70B-Instruct"},
    deployment_config={"autoscaling_config": {"min_replicas": 1, "max_replicas": 4,
                                              "target_ongoing_requests": 32}},
    engine_kwargs={"tensor_parallel_size": 2, "kv_cache_dtype": "fp8",
                  "enable_prefix_caching": True},
)
app = build_openai_app({"llm_configs": [config]})
```

5.7 Disaggregated prefill/decode — with blast radius

Adopt when (measured): (a) TPOT SLO violations trace to prefill interference (long-prompt bursts spiking ITL despite chunked prefill), or (b) phase economics favor different SKUs (compute-dense prefill pool, bandwidth-dense decode pool).

```
bash
```

```
# vLLM + NIXL KV transfer. NOTE: kv_role semantics are version-sensitive – in current  
# releases the upper-level router (Dynamo/llm-d proxy) determines actual P/D roles;  
# treat these flags as connector wiring, not role enforcement. Verify against your p  
# Prefill pool:  
vllm serve MODEL --kv-transfer-config '{"kv_connector":"NixlConnector","kv_role":"kv  
# Decode pool:  
vllm serve MODEL --kv-transfer-config '{"kv_connector":"NixlConnector","kv_role":"kv
```

Blast radius and fallback (design these before enabling):

- **KV-transfer fabric partition** (NIXL/RDMA path down): router must fall back to **monolithic serving** on the decode pool (it has full weights — it can prefill, at degraded TTFT) rather than failing requests. Keep a monolithic fallback route configured and tested.
- **Prefill-pool loss**: same fallback; decode pool absorbs at reduced goodput → brownout ladder (§6.5) sheds load.
- **Decode-pool loss**: no fallback — this is your real availability domain; size N+1 there first.
- **Transfer latency** adds to TTFT: budget it (measure per-token KV transfer time over your fabric) and re-check the TTFT SLO before declaring victory.

Orchestrate with **NVIDIA Dynamo** or **llm-d**; prefix-cache-aware routing alone has produced multi-fold throughput and TTFT improvements in published case studies — figures are config-specific, so locate and verify the primary source before quoting any number (Appendix B worklist). **Mooncake** supplies cluster-wide KV pooling across DRAM/SSD/RDMA [Cited: FAST '25].

STAGE 6 — PLATFORM ENGINEERING (the layer the engine cannot give you)

The engine will do what it is told. The platform's job is to stop it being told to do something stupid. Each subsection below is a mechanism with an owner, a failure mode, and a runbook hook — the **owner-primitive model**: *Gateway* owns auth/routing/quotas; *Scheduler* owns admission/fairness; *Runtime* owns batching/KV; *Platform* owns placement/autoscaling; *SRE* owns SLOs/runbooks; *Security* owns provenance/audit.

6.1 Admission control & priority classes (owner: gateway + scheduler)

Mechanism — pre-engine checks, in order:

1. **Authn** → **tenant resolution** → **budget lookup** (tenant TPM/RPM, concurrent-request cap).

2. **Token budget check:** $\text{count_tokens}(\text{input}) + \text{max_tokens} \leq \text{tenant_request_budget}$ AND $\leq \text{model context policy}$. Reject oversize with 400 (policy) — never let the engine discover it.
3. **Priority class assignment:** map tenant/route $\rightarrow$ {Critical, Standard, Sheddable}; propagate to the EPP criticality field.
4. **Fleet-state gate:** if fleet is past its goodput knee (queue depth or KV pressure signal), **shed Sheddable with 429 + Retry-After**, queue Standard with a bounded deadline, always admit Critical until its reserved capacity is exhausted. **Retry discipline:** clients (and any retrying gateway tier) must honor Retry-After with **jittered exponential backoff** — synchronized retries at the Retry-After instant re-create the spike exactly as the brownout ladder steps down, turning one overload into an oscillation. Publish the backoff requirement in the API contract and reject pathological retry patterns at the rate limiter.
5. **Cancellation:** client disconnect propagates to engine abort — abandoned generations are pure waste.

Worked queue interaction: a 128K-context Sheddable request arriving at $\text{kv_cache_usage_perc} = 0.88$ is the canonical preemption-storm trigger. Admission rejects it (predicted KV ≈ 40 GiB $>$ remaining budget) before it evicts twenty 4K-context Standard requests. This single check eliminates the most common shared-platform incident class.

Failure mode: budget service down $\rightarrow$ fail-static to cached budgets with conservative defaults; never fail-open on Critical capacity.

6.2 Tenant isolation & fairness (owner: scheduler + platform)

- **Isolation tiers:** dedicated replicas for Critical tenants (hard isolation) $\rightarrow$ shared pools with weighted fairness for Standard $\rightarrow$ best-effort for Sheddable.
- **Queue discipline, concretely:** shared pools run **weighted fair queueing over token-cost, not request count** — each tenant has a weight; the scheduler dequeues the tenant with the lowest $\text{served_token_cost} / \text{weight}$, where a request's cost = $\text{input_tokens} + \text{E}[\text{output_tokens}]$ (estimate from tenant history, reconcile on completion). Request-count fairness is gameable by long-context tenants; token-cost fairness is not. *Worked example — two tenants, one bursty.* Tenant A (weight 2, chat: ~ 500 in / ~ 300 out) and Tenant B (weight 1, RAG: $\sim 8,000$ in / ~ 400 out). Estimated cost per request: $A \approx 800$, $B \approx 8,400$. Both start at debt 0. B bursts 10 requests; A sends 4.
 - Dequeue order is by debt/weight : B's first request runs ($0/1 = 0$, tie broken arbitrarily), B's debt $\rightarrow 8,400/1 = 8,400$. Now every A request ($0/2 = 0$) runs before B's second — A's four requests complete (debt $3,200/2 = 1,600$) before B runs again. Net effect: **one B request admitted per ~ 10.5 A-requests-worth of tokens**, exactly the 2:1 weight ratio over token cost — B's burst cannot starve A, and A's high request count cannot starve B either.

- **Prediction error:** suppose B's request actually generates 4,000 output tokens, not 400 (agentic blow-up). At completion, reconciliation adds the 3,600-token delta to B's debt — B's *next* dequeue is pushed further out, so the error self-corrects within one request. The dangerous direction is *under*-estimation at **admission** (reserve accounting admitted a request 10× its predicted size): bound it with per-request max_tokens caps (§6.1) so the worst-case error is capped, and track $\frac{|\text{actual} - \text{predicted}|}{\text{predicted}}$ per tenant — persistently high values mean that tenant needs a tighter cap or its own pool.
- Debt decays (e.g., halved per minute of inactivity) so a tenant idle for an hour doesn't return with infinite priority.
- **Starvation bound (aging):** no admitted request waits more than max_queue_wait (e.g., 2× its class's TTFT SLO) — a request exceeding it gets a priority boost regardless of tenant weight, or a 503 with `Retry-After` if the deadline is unmeetable. Bounded-deadline queues turn "stuck behind a heavy tenant" from an incident into a defined behavior.
- **Reserve partitioning, with numbers:** a shared pool is split as **Critical reserve** ≥ 30% of knee capacity (admission for Critical succeeds even at full Standard load), **Standard** 50–60%, **Sheddable** ≤ 10–20% (first shed, never reserved). Reserves are admission-time accounting against the goodput knee (§2.5), not GPU partitions — the engine still batches everything together.
- **Token-budget accounting under streaming cancellation:** charge tenants for tokens **generated**, not tokens delivered — a client that disconnects mid-stream still consumed decode capacity until cancellation propagated. Reconcile on engine-side completion records, not gateway-side delivery counts; the delta between the two is your cancellation-waste metric (high values mean slow cancellation propagation, §1.3).
- **Multi-turn accounting:** with prefix caching, turn N re-sends the conversation but pays prefill only for the uncached suffix. Bill cached input tokens at a discounted rate (they cost VRAM residency, not compute) and meter per-**session** token budgets alongside per-request ones — per-request budgets alone let a tenant accumulate unbounded session context.
- **Per-tenant KV reservation:** cap any tenant's share of $\text{kv_cache_usage_perc}$ (engine-level where supported; otherwise enforced via per-tenant concurrency × max-context at admission).
- **Prefix-cache namespacing:** shared system prompts create both an isolation question (tenant A's hot prefix evicting tenant B's) and a **chargeback question**. Policy options: namespace caches per tenant (clean, less reuse) or share with metered attribution (cache-hit tokens billed at the discounted rate to the *consumer*, VRAM carry attributed pro-rata to hit counts).
- **Noisy-neighbor detection & routing:** classify requests by predicted cost (context length percentile); route heavy contexts (e.g., >32K) to a dedicated **heavy pool**. Signals: per-tenant

preemption count, per-tenant TTFT-miss spike, fairness-debt
(`served_token_cost/weight`) divergence across tenants).

6.3 LoRA / multi-adapter serving (owner: platform)

Running a distinct 70B deployment per fine-tune is financially ruinous; multiplex adapters on shared base-model pools:

```
bash

# vLLM multi-LoRA (flags version-sensitive)
vllm serve meta-llama/Llama-3.1-70B-Instruct \
  --enable-lora \
  --max-loras 8 \
  --max-lora-rank 64 \
  --lora-modules tenant-a=/adapters/tenant-a tenant-b=/adapters/tenant-b
```

Design points: adapter **hot-swap cost** (host→device copy on activation — keep hot adapters resident, LRU-evict cold); **rank-mix fragmentation**: multiplexing adapters of different ranks (e.g., rank-16 and rank-64 in one engine) fragments the adapter VRAM pool and disrupts continuous-batching efficiency — standardize ranks per pool where possible, or segregate by rank tier; **adapter-aware routing** (Inference Extension LoRA locality: route tenant-a to replicas already holding the adapter); per-tenant adapter isolation (an adapter is tenant data — registry + signing per §6.9); eval-gate adapters like models (§6.10).

6.4 Structured output & agentic serving (owner: runtime + gateway)

Grammar-constrained decoding (xGrammar, Outlines, llguidance; SGLang jump-forward) enforces JSON-schema/regex validity at the sampler. Platform implications: **schema compilation is CPU work** — cache compiled grammars keyed by schema hash, or high-QPS structured traffic shifts the bottleneck host-ward; jump-forward multi-token emission improves latency on deterministic spans; failure modes to test: truncation under (`max_tokens`) mid-object (invalid JSON — gateway must detect and surface, not silently pass), EOS-bias distortions on tail fields. Agentic loops multiply request count per task — admission budgets (§6.1) must count *tokens per task*, not per request, to stop runaway agents.

6.5 Degradation modes & brownout ladder (owner: SRE + scheduler)

Define behavior *before* saturation. Ladder keyed to fleet KV pressure and goodput-knee proximity:

Trigger	Action (cumulative)
<code>kv_cache_usage_perc > 0.85</code> sustained	Stop admitting Sheddable (429 + Retry-After); alert
<code>> 0.90</code>	Cap <code>max_tokens</code> on Standard (e.g., →512); disable speculative decoding (frees draft VRAM + verification compute)
<code>> 0.95</code>	Route Standard overflow to fallback smaller model (e.g., 70B→8B with response header noting degraded tier); Critical only on primary
<code>> 0.97</code> or preemption storm	Shed Standard; Critical-only mode; page
Recovery	Step down ladder with hysteresis (re-admit one tier per stable interval — no flapping)

Every rung is a feature flag, exercised in game days — an untested brownout mode is a second incident.

6.6 DR, multi-region, and capacity contingency (owner: platform)

- **Model artifact replication:** every served model + adapter mirrored to a per-region private OCI registry; cold-start path never crosses regions or touches public internet.
- **Warm-spare strategy:** per-region N+1 warm; cross-region failover pool with weights pre-pulled (warm = minutes; cold 70B pull = tens of minutes — your RTO is dominated by weight movement).
- **RTO/RPO:** inference is stateless ($RPO \approx 0$ — conversation state lives upstream); RTO = failover detection + warm-pool admission. Set and test a number.
- **GPU-supply contingency:** correlated failures are real (driver regression across a node group, region capacity exhaustion, NCCL deadlock taking out a TP×PP group). Plan: alternate-SKU fallback configs (the same model pre-validated on A100/MI300X pools), smaller-model fallback (§6.5 ladder), and a documented request-shedding order.
- **Spot/on-demand mix:** spot for Sheddable/batch pools with graceful-drain handlers on interrupt; never for Critical decode pools. **Drain timers must fit the interruption notice:** cloud spot notices are short and hard (AWS ~120s) — the pod's `terminationGracePeriodSeconds`, preStop sleep, and longest permitted stream on a spot node must all sum inside that window (§5.2), which usually means spot pools enforce shorter `max_tokens`/stream-duration caps than on-demand pools.

6.7 Capacity & cost engineering (owner: platform + finance)

The model [Derived] — report three metrics, never one. Prompt-heavy RAG and output-heavy chat have opposite economics; a single blended number hides which one you have:

```
$/1M INPUT  tokens at SLO  = 1e6 × hourly_platform_cost / (3600 × SLO_input_tok_s)
$/1M OUTPUT tokens at SLO  = 1e6 × hourly_platform_cost / (3600 ×
SLO_output_tok_s)
$/1M TOTAL  tokens at SLO  = 1e6 × hourly_platform_cost / (3600 × SLO_total_tok_s)

where total_hourly_platform_cost = Σ GPU_hourly × replicas
                                + router/CPU + storage + egress + observability
and the goodput rates decompose by: prompt/output mix, prefix-cache hit rate,
speculative acceptance rate, admission rejection rate.
```

Allocation note: input tokens consume prefill compute; output tokens consume decode bandwidth; cached input tokens consume neither (VRAM residency only). A cost model that bills them identically will mis-price RAG by an order of magnitude.

Worked example [Derived — illustrative prices, substitute yours; output-token framing]:

- 4 replicas × 2×H100 @ \$2.50/GPU-hr = \$20/hr; +\$2/hr platform overhead → **\$22/hr**.
- Measured SLO goodput at the knee: 3,000 **output** tok/s fleet-wide (alongside, say, 12,000 input tok/s at a 4:1 prompt:output mix → also report \$0.51/1M input, \$0.41/1M total).
- → $\$22 / (3600 \times 3000) \times 10^6 \approx$ **\$2.04 per 1M output tokens at SLO**.
- **FP8 + prefix caching scenario:** FP8 halves GPU count for the same goodput (2 replicas × 2 GPU = \$12/hr) and a 40% prefix-hit rate lifts effective goodput $\sim 1.5\times \rightarrow \$12 / (3600 \times 4500) \times 10^6 \approx$ **\$0.74/1M output** — a 2.8× reduction from two software changes. This is the shape of argument that defends a capacity plan to finance.
- **Utilization cliff:** running at 60% of knee for latency safety raises \$/1M by 1.67×; pushing to 95% saves money until one burst triggers the brownout ladder. Price the headroom explicitly.

The hidden-cost ledger (the line items that surprise finance later):

- **Critical reserve idle cost:** the §6.2 reserve ($\geq 30\%$ of knee for Critical) is capacity you pay for whether or not Critical traffic arrives — it is an availability premium, priced as such, not "waste."
- **Disaggregation network cost — with magnitude, parameterized by KV dtype:** cross-zone KV transfer is metered egress in cloud, and the numbers are not small. $\text{KV_transfer_GiB} = \frac{\text{context_tokens} \times \text{KV_bytes_per_token}(\text{kv_dtype})}{2^{30}}$. For Llama-3-70B at 10K context [**Derived, §2.1**]: BF16 KV (0.31 MiB/token) $\approx$ **3.0 GiB**; FP8 KV (0.155 MiB/token) $\approx$

1.5 GiB — at a typical ~\$0.01/GB cross-AZ rate, ~\$0.03 vs ~\$0.015 of pure network tax per request, at scale often exceeding the marginal GPU cost of the prefill itself. (FP8 KV halves the egress line as well as the VRAM line — another reason it's the §4.4 default.) Keep prefill and decode pools **zone-local** (same-AZ transfer is typically free) and treat any cross-zone disaggregation design as having failed this line item until proven otherwise.

- **Regional duplication:** every active region multiplies (replicas × model variants × adapters); DR warm spares (§6.6) are additional.
- **Cache-hit drift:** the \$0.74 figure above assumes a 40% hit rate *that decays* as tenant mix and prompt templates evolve — monitor hit rate as a cost SLI and re-forecast quarterly.
- **Tokenizer variance:** tokens-per-character differs by tokenizer and language; cross-model cost comparisons must normalize or they're comparing different units.
- **Failed canaries and rollbacks:** each blocked promotion (§6.10) consumed eval-suite GPU time and canary capacity — budget a release-engineering overhead line (~2-5% of fleet in active-development phases).
- Add: reserved-vs-on-demand blend (commit the base, burst on demand) and the break-even point at which a smaller fallback model serves Standard traffic cheaper than scaling the large one.

6.8 Audit, compliance, and prompt-data policy (owner: security)

Request/response **audit schema**: timestamp, `X-Request-ID`, tenant, model+revision, adapter, token counts, latency metrics, admission decision, criticality, completion status — *content logging is a separate, explicit policy decision* (PII redaction pipeline, retention clock, encryption at rest, access controls). Map controls to your regimes (SOC 2, HIPAA, EU AI Act transparency duties); keep the model inventory (§6.9) as the system-of-record auditors see first.

Transport security / zero-trust: gateway↔engine traffic is internal but not exempt — regulated environments require **mTLS** on it, typically via service mesh (Istio/Linkerd, sidecar or ambient) since inference engines don't terminate TLS well themselves. The §5.2 NetworkPolicy is the floor (who may connect); mesh mTLS adds identity and encryption (proof of who connected). Budget the mesh's latency overhead into the TTFT SLO — measure it, don't assume it's free.

6.9 Model governance & provenance (owner: security + platform)

Pipeline: **private OCI registry** (ORAS/registry-native model artifacts) → **sign** (sigstore/cosign) → **scan** (pickle/code-exec scanning for non-safetensors; CVE scan on serving images) → **model card** with provenance, license, eval results stored alongside the artifact → **promotion pipeline** dev→staging→prod gated on §6.10 evals. Runtime pulls **only** signed tags from the internal registry; admission of an unsigned artifact is a build-time failure, not a runtime discovery.

6.10 Eval gates & release engineering (owner: platform + quality)

CI artifact, not a one-liner: every model/quant/adapter/engine-version change runs the gate —

lm-eval baseline (GSM8K, MMLU, IFEval) + long-context (RULER) + tool-calling (BFCL) + domain corpus + **sampled production-traffic replay** — and **blocks promotion** on regression beyond the per-metric error budget. Canary rollout 5%→50%→100% gated on **goodput and quality signals** (structured-output validity rate, tool-call success rate), with automated rollback. A quantized canary whose BFCL drops 5 points rolls back without a human in the loop.

6.11 Incident response (owner: SRE)

Severity matrix (SEV1 = Critical-tenant SLO breach or fleet-wide goodput collapse; SEV2 = brownout rung ≥ 2 engaged; SEV3 = single-replica/degraded redundancy). Paging policy tied to the alert set (§2.4). Postmortem template links: triggering signal → runbook used → ladder rungs engaged → capacity model delta → action items with owners. The runbook table below is the entry point:

Signal	Diagnosis	Action
TTFT ↑, TPOT stable	queue/capacity	admission first (shed Sheddable), then scale out
TPOT ↑	decode pressure: long ctx, KV saturation, prefill interference	lower <code>max-num-batched-tokens</code> ; FP8 KV; evaluate disaggregation
Preemptions / KV evictions / recompute events > 0	KV exhaustion	brownout ladder §6.5; reduce <code>max-model-len</code> / <code>max-num-seqs</code> . (vLLM V1 note: <code>num_requests_swapped</code> / <code>cpu_cache_usage_perc</code> are legacy swap-era metrics — rely on preemption/recompute counters unless your pinned profile still exposes swap)
OOM on spike	over-committed budget	recompute §2.1 with headroom; lower <code>gpu-memory-utilization</code>
Throughput collapse post-deploy	quant/spec-decode regression	auto-rollback; check α and eval deltas
XID / ECC errors	GPU hardware fault	cordon + drain; <code>dccgmi diag -r 2</code> ; compare busbw to §5.4 baseline
NCCL hang in TP/PP group	fabric or peer failure	whole parallel group is the failure domain — restart group, fail over to spare; check rail health

APPENDIX A — Tools / Tech / Framework Matrix

Layer	Primary	Alternatives / notes
Engine	vLLM	SGLang, TensorRT-LLM, llama.cpp, LMDeploy
Gateway / admission	Envoy AI Gateway, Gateway API Inference Extension (kgateway)	LiteLLM, NGINX
K8s serving	KServe (LLMInferenceService), llm-d	Raw Deployments, KubeRay
Autoscaling	KEDA (Prometheus triggers) + admission-led shedding	HPA custom metrics; Dynamo planner
Distributed	Ray Serve, Dynamo	vLLM run_cluster (TP×PP), sgl-router
KV transfer / pooling	NIXL, LMCache	Mooncake
Quantization	llm-compressor (FP8), AutoAWQ	GPTQModel, TRT-LLM quant, GGUF
Speculative	EAGLE-3	Medusa, n-gram/prompt-lookup
Structured output	xGrammar (SGLang/vLLM)	Outlines, llguidance
Multi-adapter	vLLM --enable-lora + EPP LoRA locality	per-tenant pools
Profiling	Nsight Systems/Compute, PyTorch Profiler	torch.cuda memory snapshot, py-spy
GPU telemetry	DCGM exporter + Grafana	nvidia-smi dmon, node-exporter
Benchmarking	vllm bench serve, genai-perf/AIPerf	k6/locust (gateway), nccl-tests (fabric)
Eval gating	lm-eval + RULER + BFCL + IFEval + traffic replay	domain evals; CI promotion gates
Governance	private OCI + cosign + scanning	model cards, promotion pipeline
Fairness / quota	Kueue cohorts, EPP criticality	per-tenant pools

APPENDIX B — Claims Register (the resolve-or-remove worklist)

Per §0.3: before external publication, every [Vendor]/[Directional] row below is resolved (primary source + config located, tag upgraded) or the claim is removed from the body. Internal adoption may proceed with rows open, but each open row is a known debt.

Claim	Tag	Status / source to confirm
PagedAttention 2-4× vs FasterTransformer/Orca	[Cited]	Kwon et al., SOSP '23, arXiv:2309.06180, paper config
FA3 ~75% util FP16 / ~1.2 PFLOPS FP8 on H100	[Cited]	Shah et al., arXiv:2407.08608; benchmark- specific; active kernel path must be confirmed per profile
H100 ridge ≈295 FLOPs/byte FP16	[Derived]	989 TFLOPS / 3.35 TB/s — SXM5 variant
RadixAttention up to 6.4× prefix- heavy	[Cited→Directional]	SGLang paper; vLLM has since added prefix caching — re-benchmark
SGLang jump-forward ~2× structured-output latency	[Vendor]	locate primary SGLang benchmark + config, or remove number
EAGLE-3 3-6.5× low batch	[Cited→Directional]	EAGLE-3 paper; production gains lower, workload-dependent
TGI maintenance mode	[Cited — resolved]	HF Inference Endpoints docs: maintenance mode as of 2025-12-11, vLLM/SGLang recommended (verified against official docs as of 2026-06-10; re- verify per profile)
llm-d / prefix-aware routing case- study gains	[Vendor — number removed from body]	locate the primary case study + config before restoring any figure
NIXL (kv_role) semantics	[Cited — resolved]	vLLM NixlConnector docs: (kv_role) is effectively a placeholder; actual P/D roles set by the upper-level proxy — matches §5.7 wording
KServe (LLMInferenceService) CRD	[Cited — resolved]	KServe generative-inference docs confirm the CRD and router/scheduler features; field shapes remain profile-pinned
vLLM metric names used in §2.4	[Cited — resolved at family level]	vLLM metrics design docs confirm the metric family (running/waiting, kv_cache_usage, prefix-cache, TTFT/ITL histograms); exact names re-verified per profile

Claim	Tag	Status / source to confirm
MI300X ~37-66% of H100 realized	[Directional, dated]	SemiAnalysis-era; ROCm/vLLM moving fast — re-benchmark current stack
Blackwell FP4 production guidance (mixed precision)	[Vendor]	NVIDIA materials; validate per-model accuracy on your evals
H200 141 GB / 4.8 TB/s; B200 192 GB; MI325X 256 GB / 6 TB/s	[Derived from specs]	vendor spec sheets
NVLink4 900 GB/s vs PCIe Gen5 ~128 GB/s	[Derived from specs]	bidirectional figures
Two all-reduces/layer in Megatron TP forward	[Cited]	Shoeybi et al., Megatron-LM
Mooncake KVCache-centric disaggregation	[Cited]	FAST '25; per-version behavior of NIXL/LMCache connectors still needs profile pinning
All CLI flags, CRD shapes, metric names, connector roles, kernel paths	—	version-sensitive; re-verify against the §0.1 profile

APPENDIX C — Co-Hosting Boundary: Embeddings, Rerankers, Multimodal

These workloads share clusters with LLM serving but break several assumptions this playbook makes; this appendix marks where the model changes rather than fully designing it.

Embeddings & rerankers. Tiny models, no decode phase, no KV cache — pure batched prefill-like compute with millisecond budgets. Implications: (a) **don't co-locate them on LLM decode GPUs** — their bursty compute spikes interfere with TPOT, and they pack well onto fractional/MIG slices or L4-class GPUs instead; (b) the §2 metrics model collapses to latency + QPS (no TTFT/TPOT split, no goodput-knee asymmetry); (c) in RAG pipelines they sit *upstream* on the request path, so their P99 adds directly to the user-perceived TTFT budget — allocate it explicitly in the SLO. Serve via the same gateway and admission layer (cheap per-request but high-QPS, so request-rate quotas matter more than token budgets).

Multimodal (vision-language, audio). Breaks the capacity math in three places: (a) the **encoder** (ViT/audio) adds a per-request compute stage before prefill whose cost scales with media size, not token count — admission budgets must count media units, not just tokens; (b) image tokens inflate effective context (hundreds to thousands of tokens per image), so the §2.1 KV arithmetic

needs the *post-projection* token count, which is model-specific; (c) media payloads inflate request size limits, gateway buffering, and audit-log policy (media retention is a separate compliance decision from text). Engine multimodal paths are newer and more version-volatile than text serving — pin and validate per profile with extra suspicion.

Full treatment of both — encoder pool sizing, media-aware admission, multimodal eval gates — is a candidate companion alongside the MoE topology document.

APPENDIX D — Canonical Evidence Bundle & Machine-Validation Ledger

This document includes one canonical evidence bundle for the artifacts validatable in an authoring environment; the paired repository contains the complete rerunnable artifacts, logs, and GPU-dependent evidence. Each entry carries (profile / date / artifact-hash / operator); entries marked **BLOCKING** are required-empty until a GPU profile run fills them, and they are the precise boundary between this reference and a production platform standard.

D.1 Executed evidence (real runs, this bundle)

yaml

```
evidence_bundle:
  bundle_date: "2026-06-10"
  operator: "authoring environment (CPU-only; no GPU profile available)"
  entries:
    - artifact: gateway (§1.3, extracted verbatim)
      hash: d6fd4cd3fe5a
      method: pytest, FastAPI TestClient – 5/5 passed
      evidence: |
        test_unknown_alias_404 PASSED
        test_max_tokens_admission_400_before_any_network PASSED
        test_route_round_robin_cycles_pool PASSED
        test_alias_translation_table_maps_to_served_name PASSED
        test_route_raises_http_exception_not_crash PASSED
      claim_validated: "admission rejects over-budget and unknown-model requests BEFORE"
    - artifact: capacity helper (§2.1)
      hash: 64f95d554796
      method: live execution against microsoft/Phi-3-mini-4k-instruct
      evidence: |
        param source: safetensors-index
        checkpoint size: 7.1 GiB
        kv/token (fp8) = 0.19 MiB   KV@16x4096 = 12.0 GiB   TOTAL+headroom = 22.9 GiB
      claim_validated: "safetensors-index path resolves checkpoint bytes; KV/token mat"
    - artifact: manifest set (§5.2) + all YAML blocks
      method: static parse – 6 blocks / 10 documents
      evidence: "all parse as valid YAML; calculator and gateway code compile"
      claim_validated: "structural validity only – NOT schema conformance against pinned"
```

D.2 BLOCKING — required-empty until a filled profile run

Evidence artifact	Make target(s)	Status
Filled <code>versions.lock</code> profile (h100-vllm-kserve)	—	BLOCKING — requires team validation run
Benchmark report (§2.5 schema, real TTFT/TPOT/goodput)	<code>bench-local</code>	BLOCKING — requires GPU
Manifest schema conformance + render on pinned stack	<code>deploy-kind</code> , <code>lint-k8s</code>	BLOCKING — requires pinned cluster
Live PromQL returns against a scraping vLLM	<code>validate-promql</code>	BLOCKING — requires live engine

Evidence artifact	Make target(s)	Status
Gateway under concurrency + cancellation propagation	<code>test-gateway</code> , <code>test-</code> <code>cancel-stream</code>	BLOCKING — unit-level passed (D.1); load-level open
Game-day bundle: brownout, quota, XID, KV-pressure, rollback	<code>test-brownout</code> ... <code>test-rollback</code>	BLOCKING — requires staging fleet

Evidence statements (the per-section pattern, used once a profile fills the table): *"Validated on profile `<name>` via `<targets>` on `<date>` by `<operator>`; full logs at `/benchmarks/runs/<sha>`."* The bundle embeds proof of one canonical pass; the repo holds exhaustive outputs for re-validation, drift checks, and incident follow-up. Do not paste full logs or transcripts into this document — the repo proves, the playbook teaches.

APPENDIX E — Caveats

- Vendor "up to" figures are ceilings from vendor-optimal configs; the body text treats them as footnotes, not planning numbers.
- Speculative decoding and disaggregation are conditional wins with explicit preconditions and rollbacks (§4.3, §5.7).
- Goodput is gameable (delayed token delivery smooths tails) — monitor P99 and token-level SLOs alongside it.
- The 405B/671B MoE multi-node topology companion (NVLink-domain planning, rail-optimized IB design, NCCL/RCCL tuning, wide-EP) is the committed next document; §5.4–5.5 carry the interim guidance.
- All VRAM math is a planning estimate; the §2.1 calculator plus measured headroom on your stack is the source of truth.